ASSIGNMENT-16.3

Roll no:2303A52074
Batch-37

Task 1 – Hospital Management Database

Prompt

-- Create schema and queries for a Hospital Management System with tables
-- Doctors, Patients, and Appointments including constraints and analytical queries.

Code

ai 16.3 > hospital.sql > ...

▷Run on active connection | ☰Select block

```sql
-- Create schema and queries for a Hospital Management System with tables
-- Doctors, Patients, and Appointments including constraints and analytical queries.
-- Create the database
CREATE DATABASE IF NOT EXISTS ai_lab16;
USE ai_lab16;
-- Create Doctors table
CREATE TABLE Doctors (
    doctor_id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    specialty VARCHAR(50) NOT NULL,
    phone VARCHAR(15) NOT NULL
);
-- Create Patients table
CREATE TABLE Patients (
    patient_id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    age INT NOT NULL
);
-- Create Appointments table
CREATE TABLE Appointments (
    appointment_id INT AUTO_INCREMENT PRIMARY KEY,
    doctor_id INT,
    patient_id INT,
    appointment_date DATE NOT NULL,
    FOREIGN KEY (doctor_id) REFERENCES Doctors(doctor_id),
    FOREIGN KEY (patient_id) REFERENCES Patients(patient_id)
);
-- Insert sample data into Doctors
INSERT INTO Doctors (name, specialty, phone) VALUES
('Dr. Smith', 'Cardiology', '123-456-7890'),
('Dr. Johnson', 'Neurology', '234-567-8901'),
('Dr. Williams', 'Pediatrics', '345-678-9012');
-- Insert sample data into Patients
INSERT INTO Patients (name, age) VALUES
('Alice', 30),
('Bob', 45),
('Charlie', 10);
-- Insert sample data into Appointments
INSERT INTO Appointments (doctor_id, patient_id, appointment_date) VALUES
(1, 1, '2024-07-01'),
(2, 2, '2024-07-02'),
(3, 3, '2024-07-03');
-- Query to find all appointments with doctor and patient names
SELECT a.appointment_date, d.name AS doctor_name, p.name AS patient_name
FROM Appointments a
JOIN Doctors d ON a.doctor_id = d.doctor_id
JOIN Patients p ON a.patient_id = p.patient_id;
-- Query to find the number of appointments for each doctor
SELECT d.name AS doctor_name, COUNT(a.appointment_id) AS appointment_count
FROM Doctors d
LEFT JOIN Appointments a ON d.doctor_id = a.doctor_id
GROUP BY d.doctor_id;
-- Query to find patients with appointments in the next 7 days
```

```sql
-- Query to find patients with appointments in the next 7 days
SELECT p.name AS patient_name, a.appointment_date
FROM Patients p
JOIN Appointments a ON p.patient_id = a.patient_id
WHERE a.appointment_date BETWEEN CURDATE() AND DATE_ADD(CURDATE(), INTERVAL 7 DAY);
-- Query to find doctors with no appointments
SELECT d.name AS doctor_name
FROM Doctors d
LEFT JOIN Appointments a ON d.doctor_id = a.doctor_id
WHERE a.appointment_id IS NULL;
-- Query to find the average age of patients for each doctor
SELECT d.name AS doctor_name, AVG(p.age) AS average_age
FROM Doctors d
JOIN Appointments a ON d.doctor_id = a.doctor_id
JOIN Patients p ON a.patient_id = p.patient_id
GROUP BY d.doctor_id;
```

Output

Tables_in_ai_lab16

Filter...

- appointments
- doctors
- patients

doctor_id	name	specialty	phone
1	Dr. Smith	Cardiology	123-456-7890
2	Dr. Johnson	Neurology	234-567-8901
3	Dr. Williams	Pediatrics	345-678-9012

SELECT * FROM Doctors; | **SELECT * FROM Patients;** | SELECT * FROM Appointments;

patient_id	name	age
1	Alice	30
2	Bob	45
3	Charlie	10

SELECT * FROM Doctors; | SELECT * FROM Patients; | **SELECT * FROM Appointments;**

appointment_id	doctor_id	patient_id	appointment_date
1	1	1	2024-07-01
2	2	2	2024-07-02
3	3	3	2024-07-03

List all appointments for a specific doctor.

appointment_date	doctor_name	patient_name
2024-07-01	Dr. Smith	Alice
2024-07-02	Dr. Johnson	Bob
2024-07-03	Dr. Williams	Charlie

Retrieve patient history by patient ID

patient_id	patient_name	appointment_date	doctor_name
Filter...	Filter...	Filter...	Filter...
1	Alice	2024-07-01	Dr. Smith

Count total patients treated by each doctor

doctor_name	total_patients
Filter...	Filter...
Dr. Smith	1
Dr. Johnson	1
Dr. Williams	1

Explanation

In this assignment, I designed a database of Hospital Management System in SQL. Then I developed a database and called it, ai_lab16. I developed three tables, Doctors, Patients, and Appointments using prompt . The arrangements are represented in the tables in which the information is stored concerning the doctors, patients, and their appointments, and relationships are anchored with the help of the foreign keys. Once the tables were made, I entered sample data in the tables. Subsequently, made queries to obtain patient history based on patient ID and also to get the number of patients treated by each doctor. Lastly, to test the query results and tables,used SELECT statements.

Task 2 – Real-Time Application: Online Shopping Database

Prompt

-- Create schema and queries for an Online Shopping Database with tables
-- Users, Products, Orders, and OrderDetails including relationships
-- and analytical queries**.**

Code

```
Run on active connection | Select block
-- Create schema and queries for an Online Shopping Database with tables
-- Users, Products, Orders, and OrderDetails including relationships
-- and analytical queries.
-- Create the database
CREATE DATABASE IF NOT EXISTS ecommerce;
USE ecommerce;
-- Create Users table
CREATE TABLE Users (
    user_id INT AUTO_INCREMENT PRIMARY KEY,
    username VARCHAR(50) NOT NULL UNIQUE,
    email VARCHAR(100) NOT NULL UNIQUE,
    password VARCHAR(255) NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
-- Create Products table
CREATE TABLE Products (
    product_id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    description TEXT,
    price DECIMAL(10, 2) NOT NULL,
    stock INT NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
-- Create Orders table
CREATE TABLE Orders (
    order_id INT AUTO_INCREMENT PRIMARY KEY,
-- Create OrderDetails table
CREATE TABLE OrderDetails (
    order_detail_id INT AUTO_INCREMENT PRIMARY KEY,
    order_id INT NOT NULL,
    product_id INT NOT NULL,
    quantity INT NOT NULL,
    price DECIMAL(10, 2) NOT NULL,
    FOREIGN KEY (order_id) REFERENCES Orders(order_id),
    FOREIGN KEY (product_id) REFERENCES Products(product_id)
);
-- Queries
-- 1. Retrieve all orders by a user
SELECT * FROM Orders WHERE user_id = 1;
-- 2. Find the most purchased product.
SELECT p.name AS product_name, SUM(od.quantity) AS total_quantity_sold
FROM OrderDetails od
JOIN Products p ON od.product_id = p.product_id
GROUP BY od.product_id
ORDER BY total_quantity_sold DESC
LIMIT 1;

--3. Calculate total revenue in a given month.
SELECT SUM(total_amount) AS total_revenue
FROM Orders
WHERE YEAR(order_date) = 2023 AND MONTH(order_date) = 10;
```

Output

Retrieve all orders by a user.

order_id	user_id	order_date	total_amount
Filter...	Filter...	Filter...	Filter...
1	1	2026-03-19 00:08:35	50000.00

Find the most purchased product.

product_name	total_quantity_s...
Filter...	Filter...
Laptop	1

Calculate total revenue in a given month.

total_revenue
Filter...
NULL

Explanation

In the E-commerce platform. I have firstly created a database named ecommerce and designed four tables namely: Users, Products, Orders and OrderDetails. The following tables are used to store customer and product information and purchase orders. The associations between tables were ensured with the help of foreign keys. Once the tables have been created, added the sample data to recreate an online shopping System. then carried out analytical observations to find orders made by a user, the most sold product, and total revenue realized in a given month. Lastly, checked the results with the help of SELECT queries..

Task 3 – Library Database

Prompt

- Create schema and queries for a Library Management System with tables
--Books, Members, and Loans including relationships and indexing for faster queries.

Code

ai 16.3 > library.sql > ...

Run on active connection | Select block

```sql
-- Create schema and queries for a Library Management System wit
-- Books, Members, and Loans including relationships and indexir

CREATE DATABASE IF NOT EXISTS LibraryDB;
USE LibraryDB;

-- Create Books table
CREATE TABLE IF NOT EXISTS Books (
    book_id INT AUTO_INCREMENT PRIMARY KEY,
    title VARCHAR(255) NOT NULL,
    author VARCHAR(255) NOT NULL,
    published_year INT,
    genre VARCHAR(100)
);

-- Create Members table
CREATE TABLE IF NOT EXISTS Members (
    member_id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(255) NOT NULL,
    email VARCHAR(255) UNIQUE NOT NULL,
    join_date DATE
);

-- Create Loans table
CREATE TABLE IF NOT EXISTS Loans (
    loan_id INT AUTO_INCREMENT PRIMARY KEY,
```

```sql
    book_id INT,
    member_id INT,
    loan_date DATE,
    return_date DATE,
    FOREIGN KEY (book_id) REFERENCES Books(book_id),
    FOREIGN KEY (member_id) REFERENCES Members(member_id)
);

-- Sample data insertion
INSERT INTO Books (title, author, published_year, genre) VALUES
('Database Systems', 'Navathe', 2016, 'Education'),
('Operating Systems', 'Silberschatz', 2018, 'Education'),
('Computer Networks', 'Tanenbaum', 2017, 'Education');

INSERT INTO Members (name, email, join_date) VALUES
('Alice', 'alice@gmail.com', '2024-01-01'),
('Bob', 'bob@gmail.com', '2024-02-01');

INSERT INTO Loans (book_id, member_id, loan_date, return_date) V
(1, 1, '2024-06-01', NULL),
(2, 2, '2024-05-01', NULL);

```

```sql
▷Run on active connection | ≡Select block
-- Retrieve all books currently issued
SELECT b.title, m.name, l.loan_date
FROM Loans l
JOIN Books b ON l.book_id = b.book_id
JOIN Members m ON l.member_id = m.member_id
WHERE l.return_date IS NULL;

-- Find overdue books (loan date > 30 days)
SELECT b.title, m.name, l.loan_date
FROM Loans l
JOIN Books b ON l.book_id = b.book_id
JOIN Members m ON l.member_id = m.member_id
WHERE l.loan_date < DATE_SUB(CURDATE(), INTERVAL 30 DAY);

-- Count number of books loaned by each member
SELECT m.name, COUNT(l.book_id) AS total_books
FROM Members m
LEFT JOIN Loans l ON m.member_id = l.member_id
GROUP BY m.member_id;
```

Output

Retrieve all books currently issued.

title	name	loan_date
Filter...	Filter...	Filter...
Database Systems	Alice	2024-06-01
Operating Systems	Bob	2024-05-01

Find overdue books (loan date > 30 days).

-- Retrieve all books currently issued SELE... | -- Find overdue books (loan date > 30 da...

title	name	loan_date
Filter...	Filter...	Filter...
Database Systems	Alice	2024-06-01
Operating Systems	Bob	2024-05-01

Count number of books loaned by each member.

< | -- Find overdue books (loan date > 30 da... | -- Count number of books loaned by each...

name	total_books
Filter...	Filter...
Alice	1
Bob	1

Explanation

In this task, I created a database of Library Management System using SQL. Initially I have established a database called LibraryDB and have created three tables: Books, Members and Loans. The library has these tables that hold the information on books in the library, registered members and books borrowed by members. The tables are linked through foreign keys to ensure that there are relationships. The Loans table was also indexed on to enhance the performance of queries. I ran queries after feeding the sample data and I was able to retrieve currently issued books, locate overdue books that had been borrowed over 30 days and chosen on how many books the member had borrowed. I lastly confirmed the querieswith SELECT statements.

Task 4: Optimize Queries
Prompt
-- Analyze and optimize SQL queries using joins and indexing to improve performance**.**

Code

```
SELECT *
FROM Books
WHERE author_id IN (
    SELECT author_id
    FROM Authors
    WHERE country = 'UK'
);
```

```
-- Analyze an SQL query and optimize it for better performance.
-- Use joins instead of subqueries and suggest indexes if needed.
-- Ensure both original and optimized queries return the same result.
SELECT b.title, a.name
FROM Books b
JOIN Authors a ON b.author_id = a.author_id
WHERE a.country = 'UK';
```

Output

book_id	title	author	published_year
Filter...	Filter...	Filter...	Filter...
1	Database Systems	Navathe	2016

book_id	title	author	published_year	genre	author_id
Filter...	Filter...	Filter...	Filter...	Filter...	Filter...
1	Database Systems	Navathe	2016	Education	1

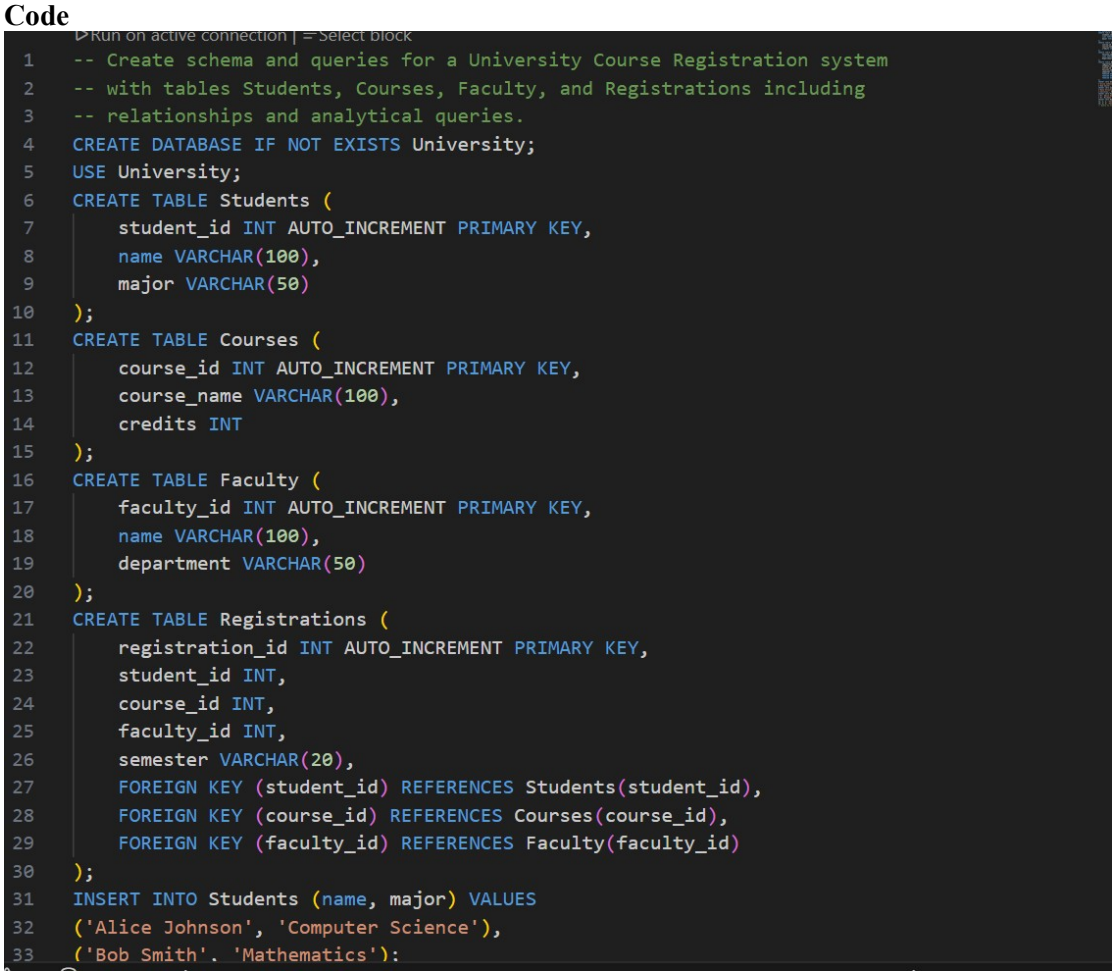
Explanation
This task involved processing and optimization of an SQL query with AI-based methodology. I have first analyzed the initial query that made a subquery to search books authored by the UK authors. Subqueries may also be slow in large data sets. I then optimized the query using the subquery with a JOIN operation that would give better performance and efficiency. I also proposed the addition of indexes on some important columns to fasten the search of data. Lastly, I ran both queries and made sure that it gave the same output and the optimized query works with bigger data sets.

Task 5: University Course Registration

Prompt

-- Create schema and queries for a University Course Registration system
-- with tables Students, Courses, Faculty, and Registrations including
-- relationships and analytical queries.

Code

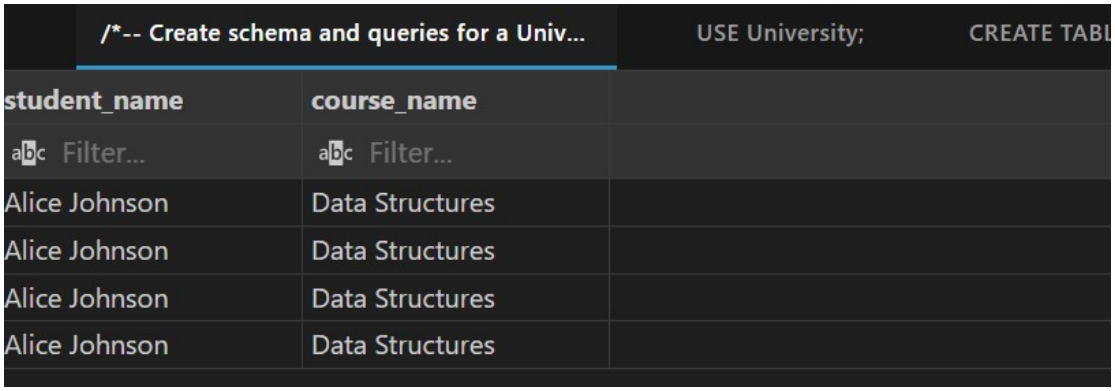


```
▷Run on active connection | = Select block
-- Create schema and queries for a University Course Registration system
-- with tables Students, Courses, Faculty, and Registrations including
-- relationships and analytical queries.
CREATE DATABASE IF NOT EXISTS University;
USE University;
CREATE TABLE Students (
    student_id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100),
    major VARCHAR(50)
);
CREATE TABLE Courses (
    course_id INT AUTO_INCREMENT PRIMARY KEY,
    course_name VARCHAR(100),
    credits INT
);
CREATE TABLE Faculty (
    faculty_id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100),
    department VARCHAR(50)
);
CREATE TABLE Registrations (
    registration_id INT AUTO_INCREMENT PRIMARY KEY,
    student_id INT,
    course_id INT,
    faculty_id INT,
    semester VARCHAR(20),
    FOREIGN KEY (student_id) REFERENCES Students(student_id),
    FOREIGN KEY (course_id) REFERENCES Courses(course_id),
    FOREIGN KEY (faculty_id) REFERENCES Faculty(faculty_id)
);
INSERT INTO Students (name, major) VALUES
('Alice Johnson', 'Computer Science'),
('Bob Smith', 'Mathematics');
```

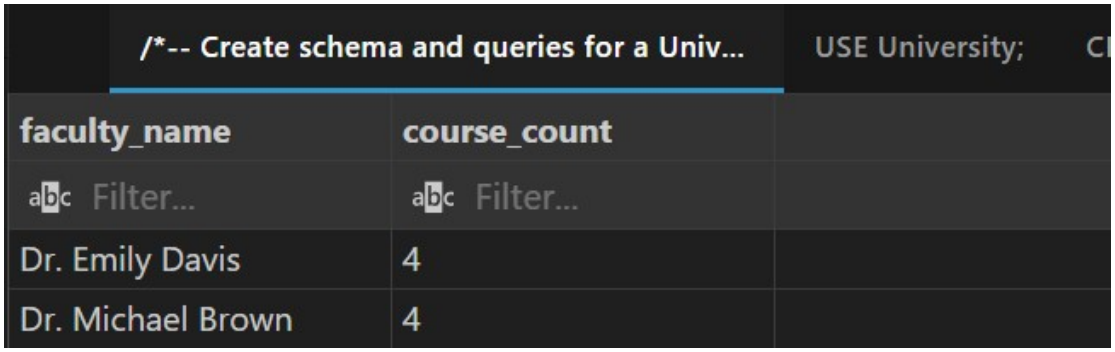
```
('Bob Smith', 'Mathematics');
INSERT INTO Courses (course_name, credits) VALUES
('Data Structures', 4),
('Calculus', 3);
INSERT INTO Faculty (name, department) VALUES
('Dr. Emily Davis', 'Computer Science'),
('Dr. Michael Brown', 'Mathematics');
INSERT INTO Registrations (student_id, course_id, faculty_id, semester) VALUES
(1, 1, 1, 'Fall 2024'),
(2, 2, 2, 'Fall 2024');

--QUERIES
-- 1. List all students enrolled in a specific course
SELECT s.name AS student_name, c.course_name
FROM Students s
JOIN Registrations r ON s.student_id = r.student_id
JOIN Courses c ON r.course_id = c.course_id
WHERE c.course_name = 'Data Structures';
-- 2. Find faculty members teaching more than 2 courses.
SELECT f.name AS faculty_name, COUNT(r.course_id) AS course_count
FROM Faculty f
JOIN Registrations r ON f.faculty_id = r.faculty_id
GROUP BY f.faculty_id
HAVING COUNT(r.course_id) > 2;
-- 3. Retrieve courses with the highest number of registrations.
SELECT c.course_name, COUNT(r.student_id) AS student_count
FROM Courses c
JOIN Registrations r ON c.course_id = r.course_id
GROUP BY c.course_id
ORDER BY student_count DESC
LIMIT 1;
```

Output

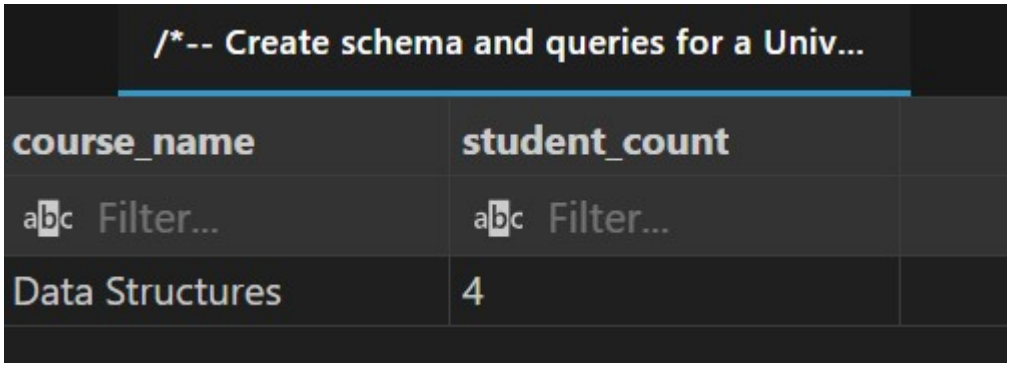
List all students enrolled in a specific course.



Find faculty members teaching more than 2 courses.



Retrieve courses with the highest number of registrations.



Explanation

In this task, I created a University Course Registration database. I used a table to manage the enrollment of students, faculties, courses and registrations. The faculty is designated to each course, and students are allowed to take several courses. Foreign keys are also used to establish relationships between the tables. I used queries to list students who are enrolled in a particular course, those who teach more than 2 courses and retrieve courses with the greatest number of registrations after I was able to insert sample data.